Mechanics Final Project: The Logistic Map

May 2020

Background

Previously in this class we have been interested in the behaviour of a system as time increases in a purely continuous manner, as this is way the time-dependence of these systems operated. However, there are real world examples in which a system is better defined by discrete times, either because that is the only way we can get an output, for example, some data from an annual event is only available for that singular time every year, or because that is the best way to analyze the system, such as it only being useful to analyze a certain animal's population at specific, significant, and singular instance in the year. This is modeled in the general equation,

$$n_{t+1} = f(n_t), \tag{1}$$

called the growth equation where the current data, n_{t+1} is determined by a function of the previous data n_t. This is often rewritten as,

$$f(n) = rn, \tag{2}$$

where r is the growth rate or parameter for the given system. The function described by equation 1 is also often called a "map", in the sense that it takes the one-dimensional value of n_t onto the next iterated time of n_{t+1}, which brings us to the focus of the given problem, the **logistic map**. The logistic map accounts for the real world parameters of these growth patterns, for example, the food available to a certain species will limit the growth of its population. This is done through a new model,

$$n_{t+1} = f(n_t) = rn_t(1 - \frac{n_t}{N}), \tag{3}$$

where N, is a constant representing the carrying capacity of the model so that as n_t increases the growth slows, and n_{t+1} would be zero if n_t were to reach N. Equation 3 can be rewritten in simplified terms if we divide it by N and simplify $\frac{n}{N}$ as a new variable x, producing,

$$x_{t+1} = f(x_t) = rx_t(1 - x_t), \tag{4}$$

where $x$is a positive number less than or equal to one.

The logistic map for equations like these contains a x value(s) for which the map is attracted to and therefore satisfies the equation

$$f(x^*) = x^*, \tag{5}$$

where x^* is a fixed point of the map, and this point can algebraically solved for as well as graphically evaluated. The fixed point of a logistic map can easily be compared to the equilibrium point of a mechanical system and therefore, it should not be surprising that we will want to evaluate the stability of these fixed points. If $|f'(x^*)| < 1$ then x* is stable, whereas if $|f'(x^*)| > 1$ then x* is unstable. The behaviour of the map and stability of such fixed points is inherently dependent on r, as as the value of r is increased we begin to see behaviour of logistic maps that will not reflect a singular attractor, x^*, rather the map shows the long-term behaviour of oscillating between two distinct points, and this motion is characterized as two-cycle. In this manner, an even greater value of r shows a pattern of stability in a four-cycle, i.e. the map is oscillating between four distinct points. With ever increasing values of r the system continues to experience period doubling, as well as reaching chaos, and this is best modeled by a **bifurcation diagram** that plots the values of x dependent on over a range of r values. The bifurcation diagram allows us to easily see where our system would descend into chaos across different r and x values, as well as the instances where period doubling occurs, which can be related to the model for nonlinear systems that experience period doubling established by Mitchell Feigenbaum,

$$(\gamma_{n+1} - \gamma_n) = \frac{1}{\delta}(\gamma_n - \gamma_{n-1}), \tag{6}$$

where δ is the constant 4.6692016.

The Problem

In this problem we will be examining a modified version of the logistic map,

$$x_{n+1} = rx(1 - \frac{x}{2}). \tag{7}$$

Part A

Look at the situation where $r = 1.5$ and your initial $x = x_0 = 0.1$.

i

Plot the value of x over the first 30 iterations. Describe its behavior.

To reference Fig. 1 and 2, using a simple while loop it is easy to plot the function Eq.7 over the first 30 iterations.This graph experiences dramatic growth, appearing to be exponential, over the first few iterations before leveling off towards the presumed attractor, i.e. x^*, around $n = 15$.

```python
r = 1.5
x = 0.1 #x_0
n = 1
xs = []
ns = []
while n <= 30:
    xs.append(x)
    ns.append(n)
    x = r*x*(1-(x/2))
    n = n + 1
```

Figure 1: Coding for part a

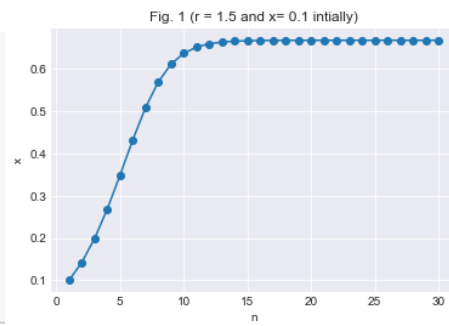

Figure 2: Behavior of the logistic map for part a

ii

Algebraically solve for all fixed points of the logistic map and evaluate their stability. Do your values agree with your plot?

To algebraically solve for the fixed stable points, we first reference Eq. 5 to solve for fixed points. In this case that means,

$$x^* = rx^*(1 - \frac{x^*}{2}), \tag{8}$$

where in solving for x^* we find that

$$x^* = 0, 2 - \frac{2}{r}. \tag{9}$$

Following our rules for stability, it is easy to see that $x^* = 0$ will be stable because $|f'(x^*)| < 1$. The second fixed point requires a small amount of calculation. For $x^* = 2 - \frac{2}{r}$, we find that $|f'(x^*)| = r - rx^*$ and therefore $|f'(x^*)| < 1$ and this fixed point i* stable as well, considering our value of $r = 1.5$ and $x^* = 2 - \frac{2}{r}$. We find that $x^* = 0.667$ and we can say that value for x^* agrees with Fig. 2 in the fact that the graph seems to settle towards 0.667, or the value of x^* for this stable fixed point.

Part B

Now look at the situation where $r = 3.2$. Again, you start at $x_0 = 0.1$.

i

Plot the value of x over the first 30 iterations. Describe its behavior. How many values does your final behavior alternate between?

Using the structure of part a code, we can easily plot this. Fig. 3 demonstrates a plot of this logistical map that grows exponentially over the first couple of iterations before settling out to oscillate, alternating between only two points, meaning the behavior is a two-cycle.

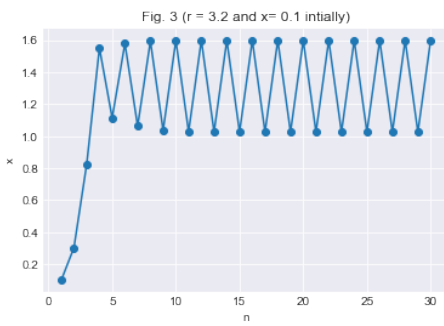

Figure 3: Two-Cycle Behavior

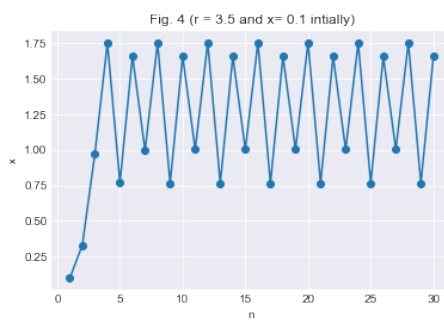

Figure 4: Four-Cycle Behavior

ii

On a different plot, plot the value of x over the first 30 iterations if $r = 3.5$ and describe the behavior. How many values does x alternate between now?

Given these new parameters, Fig 4 grows exponentially before alternating between four different values of x, i.e. the behavior is a stable four-cycle, unlike the two-cycle of Fig.3.

Part C

```python
def logistic(r, x):
    return r * x * (1 - x/2)
x = np.linspace(0, 1)
n = 1000
r = np.linspace(2.8, 4.0, n)
iterations = 1000
last = 100
x = 0.1 * np.ones(n)
for i in range(iterations):
    x = logistic(r, x)
    if i >= (iterations - last):
        plt.plot(r, x, ',k', alpha=.25)
plt.xlabel('r')
plt.ylabel('x')
plt.title("Bifurcation diagram")
plt.show()
```

Figure 5: Coding for part c

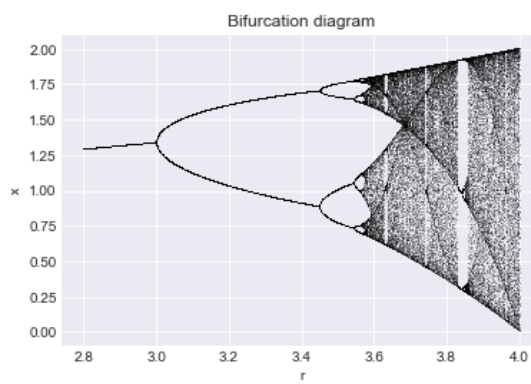

Figure 6: Bifurcation diagram

Create a bifurcation diagram of the logistic map from $r = 2.8$ to $r = 4$. Identify several regions where the system behaves chaotically and does not settle down to a late term constant behavior. The bifurcation diagram is plotted by defining our logistic map as a function of both x and r. Note that we choose a large number of iterations, between 100 and 1000, as to let the transients die out. Referencing Fig. 6, in regions $3.57 < r < 3.62$,

$3.63 < r < 3.78$, $3.78 < r < 3.82$, and $3.84 < r$ the system behaves chaotically. *Using the bifurcation diagram, in this case, to find the exact areas of chaos was tricky, and the values I found cannot be called anything more accurate than an estimation.*

Part D

Looking at your bifurcation diagram, you can see that the period doubling starts happening at a faster and faster rate. You can investigate this using the Feigenbaum relation(Eq. 6 in this case).

i

Replacing γ in Eq. 6 with r, show that Eq.6 implies that

$$r_{n+1} - r_n = \frac{K}{\delta^n}, \tag{10}$$

where K is a constant. By rearranging the whole equation and taking everything to the power of $n-1$, we can say that

$$(\frac{1}{\delta})^{n-1} = (\frac{r_{n+1} - r_n}{r_n - r_{n-1}})^{n-1}. \tag{11}$$

If $n = 1$, then $r_{n+1} - r_n = r_2 - r_1$, and we can say that

$$(\frac{1}{\delta})^{1-1}(r_2 - r_1) = r_2 - r_1, \tag{12}$$

rewriting Eq.12 in general terms, and considering Eq. 11, we can say that,

$$(\frac{1}{\delta})^{n-1}(r_2 - r_1) = r_{n+1} - r_n = (\frac{r_{n+1} - r_n}{r_n - r_{n-1}})^{n-1}(r_2 - r_1), \tag{13}$$

This holds true for $n = 2, 3$, although I will not go through those proofs here in the interest of space and length. In that case we can rewrite Eq. 13 as,

$$r_{n+1} - r_n = (\frac{1}{\delta})^{n-1}(r_2 - r_1) = \frac{\delta}{\delta^n}(r_2 - r_1) = \frac{K}{\delta^n}, \tag{14}$$

where K is a constant that encapsulates $\delta, r_2, and r_1$, and thus Eq. 10 holds true. *This proof proved a little tricky, one really needs to consider the fact that we are using certain conditions, in this case, $n = 1$ to make a general claim, that then needs to be tested under different conditions.*

ii

Using your bifurcation diagram as a guide, pinpoint to 3 decimals the points when period doubles from 1 to 2, 2 to 4, and 4 to 8. Make a plot of $ln(r_{n+1} - r_n)$ against n. The best-fit straight line to the data

has a slope equivalent of -ln(δ), where δ is the Feigenbaum constant. Find this value. This value describes how fast the period doubling cascade happens as r varies.

Looking at Fig. 6, the first three instances of period doubling occur at $r_1 = 3.000$, $r_2 = 3.455$, and $r_3 = 3.565$, as we can see from the bifurcation diagram. Plotting, in Fig. 8, the $ln(r_{n+1} - r_n)$ against n, we can see that we get a slope of -1.412.By raising e to 1.412 we should find it is equivalent to the Feigenbaum constant, δ. We find that $e^{1.412} = 4.136$. This is not the correct value of 4.669, however, given the accuracy of measuring r to three decimal places based on a bifurcation diagram, it is consistent enough.

```
y_data = np.array([np.log(3.455-3.00),np.log(3.565-3.455)])
x_data = np.array([1,2])
plt.plot (x_data, y_data, '-o')
m,b = np.polyfit(x_data, y_data, 1)
print( 'The best fit line of this data is', m)
plt.ylabel('$ln(r_{n+1}-r_n$)')
plt.xlabel('n')
plt.title('δ', fontsize = 12)
plt.show()
```

Figure 7: Code for part d

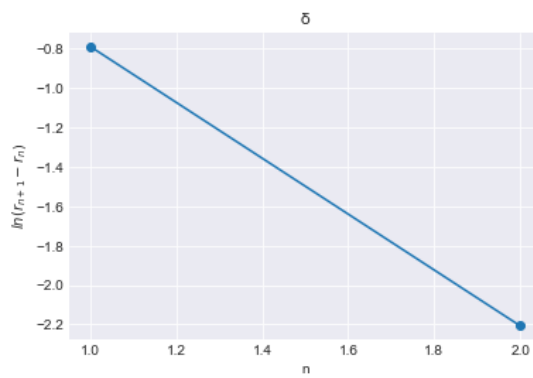

Figure 8: Equivocating our data to Feigenbaum's constant